```
.dseg ; segmento de dados
.ORG SRAM_START
    A : .BYTE 4
    B : .BYTE 4
    C : .BYTE 4

.cseg ;segmento de código

start:

    ;teste das sub_rotinas
    ;testar o init_32bits
    ;A+ 0xCCDDEEFF
    LDI R16, 0XFF
    LDI R17, 0XEE
    LDI R18, 0XDD
    LDI R19, 0XCC
    LDI XL, LOW(A)
    LDI XH, HIGH(A)
    rcall init_32bits

    LDI XL, LOW(A)
    LDI XH, HIGH(A)
    rcall zera_32bits

    ;B = 1 = 0X00000001
    LDI R16, 0X01
    LDI R17, 0X00
    LDI R18, 0X00
    LDI R19, 0X00
    LDI XL, LOW(A)
    LDI XH, HIGH(A)
    rcall init_32bits

    ; C = A -B = 0 - 1 = 0XFFFFFFFF

    LDI XL, LOW(A)
    LDI XH, HIGH(A)
    LDI XL, LOW(B)
    LDI XH, HIGH(B)
    LDI XL, LOW(C)
    LDI XH, HIGH(C)
    rcall sub_32bits


    rjmp start

;=================================================================
```

```
;init_32bits
;
;Inicializa uma variável de 32 bits apontada por X com os valores passados em R16, R17, R18 e R19
;
; Parâmetros:
;
; R16, R17, R18, R19 : Valor que deve ser utilizado na inicialização. Sendo o R16 o byte menos significativo e o R19 o mais.
;
; Registrador X:Ponteiro para a posição inicial da variável. Considere o byte menos significativo armazenado no menor endereço.
;
;===================================================================

init_32bits:

    PUSH XL
    PUSH XH

    ST X+, R16 ; fizemso store no x e já usamos um pós incremento pra guardar em "x"
    ST X+, R17
    ST X+, R18
    ST X, R19 ; não precisa incrementar a ultima

    POP XH
    POP XL

    ret

; aqui usei endereçamento indireto


;===================================================================
;zera_32bits
;
;Inicializa uma variável de 32 bits com zero.
;
;Parâmetros:
;Registrador X : Ponteiro para a posição inicial da variável.
;
;===================================================================

zera_32bits:

    ;ponteiro x vem carregado com o ndereço da variavel e ele tem 4 bits
    PUSH R16
    PUSH XL
```

```
    PUSH XH

    CLR R16
    ST X+, R16
    ST X+, R17
    ST X+, R18
    ST X, R19

    POP XH
    POP XL
    POP R16

    ret

; aqui usei endereçamento indireto

;=================================================================
;sub_32bits
;
;Subtraia duas variáveis de 32 bits e guarde em uma terceira variável.
;
;Parâmetros:
;
;   -Registrador X : Ponteiro para a posição inicial da primeira variável.
;
;   -Registrador Y : Ponteiro para a posição inicial da segunda variável, a qual será subtraída
da primeira.
;
;   -Registrador Z : Ponteiro para a posição inicial da terceira variável.
;
;=================================================================

;~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
;MANEIRA DE RESOLVER A 3 DE SUBTRAIR
;sub_32bits:
;
;    CLC
;    LDI R16, 0
;
;LOOP_SUB:
;    LD R0, X+ ; carrego qoue esta sendo apontado pro x e incrementar
;    LD R1, Y+
;    SBC R0, R1 ; SOMA E QUARRY ZERADO
;    ST Z+, R0
;
;    INC R16
;    CPI R16, 4 ; é igual a 4? ai sai
;    BRNE LOOP_SUB
```

```
;~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

sub_32bits:

    PUSH R0
    PUSH R1
    PUSH R16

    ;o in é uma instrução de registrdaor de entrada e saida, ele é como se fosse um load, um
LD
    ; depois vamos ver em detalhe o in e o out (in to trazendo pra dentro da CPU
    ;o registrador que é de entrada e saida) o out devolve. Como ld e sto
    IN R16, SREG ;ESSAS DUAS LINHAS SALVAM O SREG
    PUSH R16    ; ESSAS DUAS LINHAS SALVAM O SREG

    CLC
    LDI R16, 4

LOOP_SUB:
    LD R0, X+ ; carrego qoue esta sendo apontado pro x e incrementar
    LD R1, Y+
    SBC R0, R1 ; SOMA E QUARRY ZERADO
    ST Z+, R0

    DEC R16
    BRNE LOOP_SUB ; ele meche no z, quando chegar em 4 avaba;

    POP R16
    OUT SREG, R16
    POP R16
    POP R1
    POP R0

    ret

    ; eu podia ter escrito isso:
    ;
    ;CLC
    ;LD R0, X+ ; carrego qoue esta sendo apontado pro x e incrementar
    ;LD R1, Y+
    ;SBC R0, R1 ; SOMA E QUARRY ZERADO
    ;ST Z+, R0
    ;QUATRO VEZES, MAS ELE PEDIU PRA FAZER LOOP, PRA ECONOMIZAR
MEMORIA, ENTÃO por isso o uso do loop
```